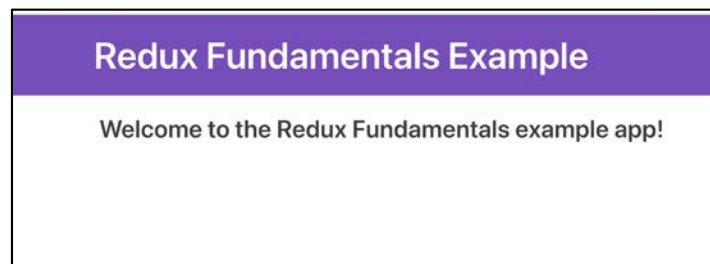


In-Class Exercise/Demo



In-Class Demo

- Let's practice some more [Redux](#) with a TODO app
- First, create a new Github repo [m6-w2-d2-demo](#) and then clone this in your WESTCLIFF folder.
- Then head over to GAP and download [m6-w2-d2-demo.zip](#), unzip and drop the content into the clone folder.
- In this challenge, we will create a ToDo app using Redux. Launch VS Code and navigate to the [m6-w2-d2-demo](#) folder.
- Let's first run [NPM install](#) to install some basic dependencies.
- Next, run [npm start](#)
- You will see a basic app.





In-Class Demo

- Now let's first take a look at the structure.
- /src
 - **index.js**: the entry point file for the application. It renders the main `<App>` component.
 - **App.js**: the main application component.
 - **index.css**: styles for the complete application
 - **/api**
 - **client.js**: a small AJAX request client that allows us to make GET and POST requests
 - **server.js**: provides a fake REST API for our data. Our app will fetch data from these fake endpoints later.
 - **/exampleAddons**: contains some additional Redux addons that we'll use later in the tutorial to show how things work



In-Class Demo

- Now Let's talk about Our App criteria:
 - The UI should consist of three main sections:
 - An input box to let the user type in the text of a new todo item
 - A list of all the existing todo items
 - A footer section that shows the number of non-completed todos, and shows filtering options
 - Todo list items should have a checkbox that toggles their "completed" status. We should also be able to add a color-coded category tag for a predefined list of colors, and delete todo items.
 - The counter should pluralize the number of active todos: "0 items", "1 item", "3 items", etc
 - There should be buttons to mark all todos as completed, and to clear all completed todos by removing them



In-Class Demo

- Now Let's talk about Our App criteria: **(continued)**
 - There should be two ways to filter the displayed todos in the list:
 - Filtering based on showing "All", "Active", and "Completed" todos
 - Filtering based on selecting one or more colors, and showing any todos whose tag that match those colors



In-Class Demo

- Complete application will look like this:

Todos

What needs to be done?

☐	Buy toys	Blue ▼	✕
☐	Buy cheese	▼	✕
☐	Buy bread	▼	✕
☐	Clean yard	▼	✕
☐	Clean car	▼	✕

Actions

Mark All Completed

Clear Completed

Remaining Todos

5 items left

Filter by Status

All

Active

Completed

Filter by Color

☐ Green

☐ Blue

☐ Orange

☐ Purple

☐ Red



In-Class Demo

First we will start by Designing the State Values

One of the core principles of React and Redux is that your UI should be based on your state. So, one approach to designing an application is to first think of all the state needed to describe how the application works.

- Conceptually, there are two main aspects of this application:
 - The actual list of current todo items
 - The current filtering options

We'll also need to keep track of the data the user is typing into the "Add Todo" input box, but that's less important and we'll handle that later.



In-Class Demo

First we will start by Designing the State Values (Continued)

For each todo item, we need to store a few pieces of information:

- The text the user entered
- The boolean flag saying if it's completed or not
- A unique ID value
- A color category, if selected

Our filtering behavior can probably be described with some enumerated values:

- Completed status: "All", "Active", and "Completed"
- Colors: "Red", "Yellow", "Green", "Blue", "Orange", "Purple"



In-Class Demo

Designing the State Structure

With Redux, our application state is always kept in plain JavaScript objects and arrays. That means you may not put other things into the Redux state - no class instances, built-in JS types like `Map` / `Set` / `Promise` / `Date`, functions, or anything else that is not plain JS data.

- First, we need an array of todo item objects. Each item should have these fields:
 - `id`: a unique number
 - `text`: the text the user typed in
 - `completed`: a boolean flag
 - `color`: An optional color category
- Then, we need to describe our filtering options. We need to have:
 - The current "completed" filter value
 - An array of the currently selected color categories



In-Class Demo

Designing Actions

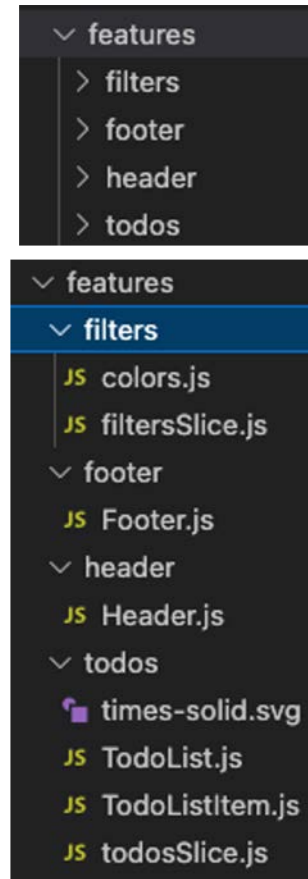
Actions are plain JavaScript objects that have a type field. As mentioned earlier, you can think of an action as an event that describes something that happened in the application.

- Add a new todo entry based on the text the user entered
- Toggle the completed status of a todo
- Select a color category for a todo
- Delete a todo
- Mark all todos as completed
- Clear all completed todos
- Choose a different "completed" filter value
- Add a new color filter
- Remove a color filter



In-Class Demo

- Next we will start splitting reducers
- Let's create a folder called `features` inside `src` and inside that we will create 4 folders `filters`, `footer`, `header`, `todos`
- Look for a folder called `files` in the root at same level as `src`
- Copy `filterSlice.js` and `colors.js` into `filters` folder
- Copy `Footer.js` file into `footer` folder
- Copy `Header.js` file into `header` folder
- Copy `times-solid.svg`, `TodoList.js`, `TodoListItem.js` and `todosSlice.js` files into `todos` folder.





In-Class Demo

- Next we will create our `store.js` in `src` folder.

```
import { configureStore } from '@reduxjs/toolkit'

import todosReducer from '../features/todos/todosSlice'
import filtersReducer from '../features/filters/filtersSlice'

const store = configureStore({
  reducer: {
    // Define a top-level state field named `todos`, handled by `todosReducer`
    todos: todosReducer,
    filters: filtersReducer,
  },
})

export default store
```



In-Class Demo

- Next we are going to update our `App.js` to make sure it includes all of our components.
- Let's import, add the import with rest of the imports on the page
`import Header from './features/header/Header'`
`import TodoList from './features/todos/TodoList'`
`import Footer from './features/footer/Footer'`
- Next we will remove `navContent` div completely.

```
<nav>
  <section>
    <h1>Redux Fundamentals Example</h1>
  </section>
</nav>
```



In-Class Demo

- Next let's add `main` tag after `nav`.
- In the `main` tag we will add section to display todo app

```
<main>
  <section className="medium-container">
    <h2>Todos</h2>
    <div className="todoapp">
      <Header />
      <TodoList />
      <Footer />
    </div>
  </section>
</main>
```



In-Class Demo

- Next we are going to update our index.js to make sure it includes all of our components.
- Let's import, add this import before `index.css` import

```
import { Provider } from 'react-redux'
```

- Add this import after `api/server`

```
import store from './store'
```

```
import { fetchTodos } from './features/todos/todosSlice'
```

- Next, we are going to dispatch todos action and add provider to our render.

```
store.dispatch(fetchTodos())
```

```
ReactDOM.render(  
  <React.StrictMode>  
    <Provider store={store}>  
      <App />  
    </Provider>  
  </React.StrictMode>,  
  document.getElementById('root')  
)
```



In-Class Demo

- Before we continue further, review all the files under `features` folder and `store.js` file
- Next we will install `redux toolkit` because we are importing from `toolkit` with in our `store.js`
- Make sure you are in the `m3-week6-d2-demo` folder, make sure your `package.json` is closed
- run `npm install @reduxjs/toolkit react-redux`
- Now, run `npm start` to start your server.



In-Class Demo

- End result
- Go ahead and interact with the application

Todos

What needs to be done?

☐ Buy toys Blue ✕

☐ Buy cheese ✕

☐ Buy bread ✕

☐ Clean yard ✕

☐ Clean car ✕

Actions

Mark All Completed

Clear Completed

Remaining Todos

5 items left

Filter by Status

All

Active

Completed

Filter by Color

☐ Green

☐ Blue

☐ Orange

☐ Purple

☐ Red



In-Class Exercise Demo

DUE:

Today 10.30PM PT.

Submission:

Post Github link on GAP Week 6 Day 2 Exercise